

C/C++ Programming Language

- Fundamentals
- Flow control statements
- Functions
- Scope rules
- Pointers
- Arrays and strings
- Memory management
- Structure and Union
- Preprocessors
- Files

Fundamentals

- Basic Program Structure
- Fundamental Data Types
- Data Types Modifiers
- Arithmetic Operators
- Logic Operators
- Bitwise Operators

Basic Program Structure

```
#include <iostream>

int main()
{
    std::cout << "Hello World!\n";

    return 0;
}
```

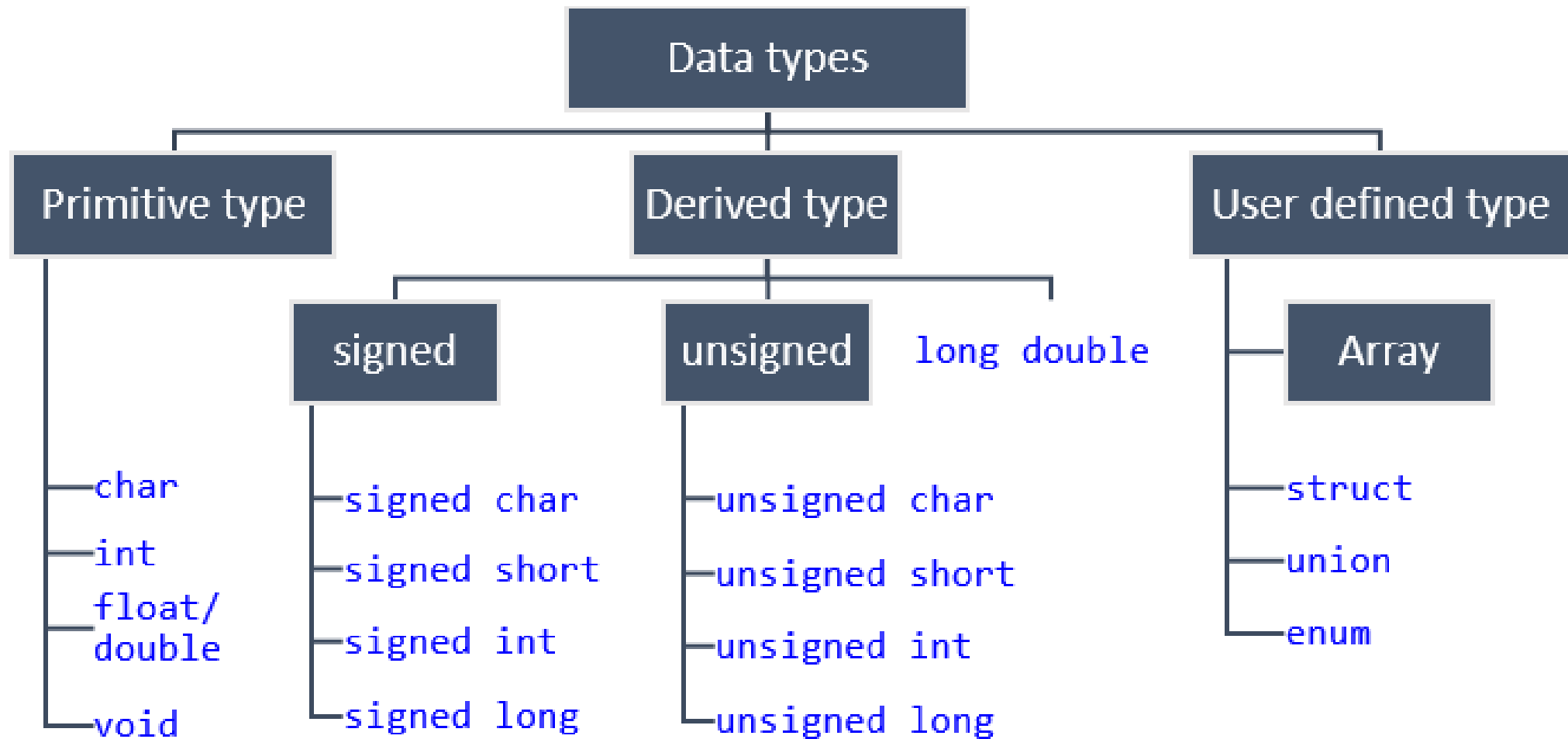
Fundamental Data Types

- Fundamental data types are the data types which are predefined in the language and can be directly used to declare a variable in C++.
- The various fundamental data types are:
 - `char` (Character)
 - `int` (Integer)
 - `float` (Floating point)
 - `double` (Double floating point)
 - `void` (valueless)

Data Type Modifiers

- signed
- unsigned
- short
- long

DATA TYPE	SIZE	RANGE
short int	2	−32,768 to 32,767
unsigned short int	2	0 to 65,535
unsigned int	4	0 to 4,294,967,295
int	4	−2,147,483,648 to 2,147,483,647
long int	4	−2,147,483,648 to 2,147,483,647
unsigned long int	4	0 to 4,294,967,295
long long int	8	−(2 ⁶³) to (2 ⁶³)−1
unsigned long long int	8	0 to 18,446,744,073,709,551,615
signed char	1	−128 to 127
unsigned char	1	0 to 255
float	4	−3.4e38 to 3.4e38
double	8	−1.7e308 to 1.7e308



Boolean (C++)

- It is the data type that is used to store two values only true or false.
- To define Boolean in C++ we use **bool** keyword.
- Size of **bool** data type is 1 byte (8 bits).

```
#include <iostream>
using namespace std;

void main()
{
    bool f;
    bool f2 = 0;    // or 1
    bool f3 = true; // or false

    cout << "f2=" << f2 << endl;
    cout << "f3=" << f3 << endl;
}
```

output:

f2=0
f3=1

Character

- It is the data type that is used to store characters like a, b, c, etc.
- To define Character in C++ we use **char** keyword.
- Size of **char** data type is 1 byte (8 bits).
- It's range is from 0 to 255.

```
#include <iostream>
using namespace std;

void main()
{
    char ch1 = 'A'; // 65
    char ch2 = 97;  // 'a'
    char ch3('C');

    cout << "ch1 = " << ch1 << " ASCII value = " << int(ch1) << endl;
    cout << "ch2 = " << ch2 << " ASCII value = " << int(ch2) << endl;
    cout << "ch3 = " << ch3 << " ASCII value = " << int(ch3) << endl;
}
```

output:

```
ch1 = A  ASCII value = 65
ch2 = a  ASCII value = 97
ch3 = C  ASCII value = 67
```


Integer

- It is the data type that is used to store integer values like 1, 2, – 1, –2, etc.
- To define Integer in C++ we use **int** keyword.
- Size of **int** data type is 4 bytes (32 bits).
- It's range is from – 2147483648 to 2147483647.

```
#include <iostream>
using namespace std;

void main()
{
    int v1 = 100;
    int v2 = 0100;
    int v3 = 0x100;

    cout << "v1 = " << v1 << endl;
    cout << "v2 = " << v2 << endl;
    cout << "v3 = " << v3 << endl;
}
```

output:

```
v1 = 100
v2 = 64
v3 = 256
```

Floating Point

- It is the data type that is used to store floating values or decimal values like 1.1, 2.1, etc.
- To define Floating point in C++ we use **float** keyword.
- Size of **float** data type is 4 bytes (32 bits).

```
#include <iostream>
#include <iomanip>
using namespace std;
```

```
int main()
{
    float f1 = 1.23, f2 = 1.23e2, f3 = 1.23E-2;

    cout << "f1 = " << f1 << " Size(Byte) = " << sizeof(f1) << endl;
    cout << "f2 = " << f2 << endl;
    cout << "f3 = " << f3 << endl;
    cout << "Max = " << numeric_limits<float>::max() << " = " << setprecision(10) << numeric_limits<float>::max() << endl;
    cout << "Min = " << numeric_limits<float>::min() << " = " << setw(20) << numeric_limits<float>::min() << endl;
    cout << "Least = " << numeric_limits<float>::lowest() << " = " << setw(20) << numeric_limits<float>::lowest() << endl;
}
```

output:

```
f1 = 1.23 Size(Byte) = 4
f2 = 123
f3 = 0.0123
Max = 3.40282e+38 = 3.402823466e+38
Min = 1.175494351e-38 = 1.175494351e-38
Least = -3.402823466e+38 = -3.402823466e+38
```

Double-Precision Floating Point

- It is the data type that is used to store floating values with higher precision.
- To define Double-Precision Floating Point in C++ we use **double** keyword.
- Size of **double** data type is 8 bytes (64 bits).

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    double d1 = 1.23, d2 = 1.23e2, d3 = 1.23E-2;

    cout << "d1 = " << d1 << " Size(Byte) = " << sizeof(d1) << endl;
    cout << "d2 = " << d2 << endl;
    cout << "d3 = " << d3 << endl;
    cout << "Max = " << numeric_limits<double>::max() << " = " << setprecision(10) << numeric_limits<double>::max() << endl;
    cout << "Min = " << numeric_limits<double>::min() << " = " << setw(20) << numeric_limits<double>::min() << endl;
    cout << "Least = " << numeric_limits<double>::lowest() << " = " << setprecision(10) << numeric_limits<double>::lowest() << endl;
}
```

output:

```
d1 = 1.23 Size(Byte) = 8
d2 = 123
d3 = 0.0123
Max = 1.79769e+308 = 1.797693135e+308
Min = 2.225073859e-308 = 2.225073859e-308
Least = -1.797693135e+308 = -1.797693135e+308
```

Valueless

- It is the data type that is used to represent a variable without any value.
- To define Valueless in C++ we use **void** keyword.
- **void** can be used as a return type of functions which indicates that function is returning nothing.

output:

nothing

```
#include <iostream>
using namespace std;

void fun(void)
{
    cout << "nothing";
}

int main()
{
    fun();
    return 0;
}
```

Arithmetic Operators

Operator	Name	Description
+	addition	It is used to add two operands.
-	subtraction	It is used to subtract two operands.
*	multiplication	It is used to multiply two operands.
/	division	It is used to numerator by the denominator.
%	modulo	It is used to give the remainder of the division.
++	increment	It is used to increment the value of the variable by 1.
--	decrement	It is used to decrement the value of the variable by 1.

Logic Operators (!, &&, ||)

! (not)	
a	!a
true	false
false	true

&& (and)		
a	b	a && b
true	true	true
true	false	false
false	true	false
false	false	false

(or)		
a	b	a b
true	true	true
true	false	true
false	true	true
false	false	false

Bitwise Operators

Operator	Name	Description
&	Bitwise-AND	Bitwise AND
	Bitwise-OR	Bitwise inclusive OR
^	Bitwise-XOR	Bitwise exclusive OR
~	Bitwise-NOT	Unary complement (bit inversion)
<<	Shift-Left	Shift bits left
>>	Shift-Right	Shift bits right

Flow Controls

- if
- If...else
- switch
- for loop
- while loop
- do...while loop
- break
- continue
- goto

if

```
if(boolean_expression)
{
    // body of if
}
```

if..else

```
if(boolean_expression)
{
    // If expression is true then execute this
}
else
{
    // If expression is false then execute this
}
```

switch

```
switch(expression)
{
    case 1:
        /* Statement/s */
        break;
    case 2:
        /* Statement/s */
        break;
    case n:
        /* Statement/s */
        break;
    default:
        /* Statement/s */
}
```

for

```
for(variable-initialization ; condition ; variable-update)
{
    // Body of for loop
}
```

while

```
while(condition)
{
    // Body of while loop
}
```

do...while

```
do
{
    // Body of do while loop
} while (condition);
```

do...while

```
do
{
    // Body of do while loop
} while (condition);
```

Pointers

```
int num  = 10;  
int *ptr = &num; // Assign address of num to ptr  
  
// You can also assign a pointer variable to another  
int *ptr1 = ptr; // Initialize pointer using another pointer
```


Functions

```
return_type function_name(parameter list)
{
    // Function body
}
```

```
#include <iostream>
using namespace std;

void printBits(unsigned int num)
{
    for (unsigned int mask = 0x0001 << (sizeof(unsigned int) * 8) - 1; mask>0; mask >>= 1)
    {
        cout << ((num & mask)?'1':'0');
    }
}

void main()
{
    printBits(12);
}
```

Preprocessors

- `#include`
- `#define`
- `#undef`
- `#ifdef`, `#endif`
- `#ifndef`, `#endif`
- `#if`, `#elif`, `#else`, `#endif`

```
include <iostream>
using namespace std;

// MACRO definition
#define COUNTRY "INDIA"

int main()
{
    // If COUNTRY is defined, print a message
#ifdef COUNTRY
        cout << "Country is defined\n";
#endif

    // If STATE is not defined, define it
#ifndef STATE
        cout << "State is not defined. Defining state. \n";
#define STATE "PATNA"
#endif

        cout << "State is: " << STATE;

        return 0;
}
```